

[6. Mapovanie hierarchie tried](#)

[Príklad - triedy grafického rozhrania](#)

[Stratégie mapovania](#)

[SINGLE_TABLE](#)

[JOINED](#)

[TABLE_PER_CLASS](#)

[Mapovanie abstraktnej rodičovskej triedy](#)

[Výber stratégie](#)

[Cvičenia](#)

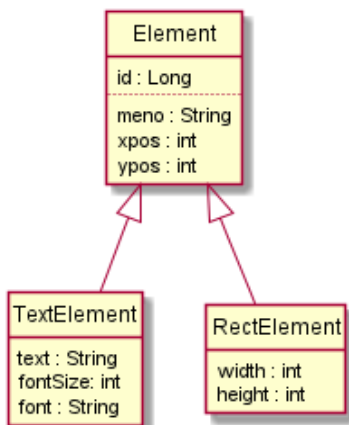
6. Mapovanie hierarchie tried

Pre mapovanie dedičnosti - hierarchie entitných tried do relačnej databázy poskytuje JPA tri stratégie.

- SINGLE TABLE - je default
- JOINED
- TABLE PER CLASS

Príklad - triedy grafického rozhrania

[Pozri projekt p61](#)



Entitné triedy

Zvolenú stratégiu pre mapovanie dedičnosti definujeme anotáciou `@Inheritance` koreňovej triedy hierarchie.

```
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
public class Element implements Serializable {
    @Id
    private Long id;
    private String name;
    private int xpos ;
    private int ypos ;
}
```

Poznámka: odvodené triedy dedia kľúčový atribút

```
@Entity
public class TextElement extends Element implements Serializable {
    private String text;
    private int fontSize;
    private String font;
}
```

```
@Entity
public class RectElement extends Element implements Serializable {
    private int width;
    private int height;
}
```

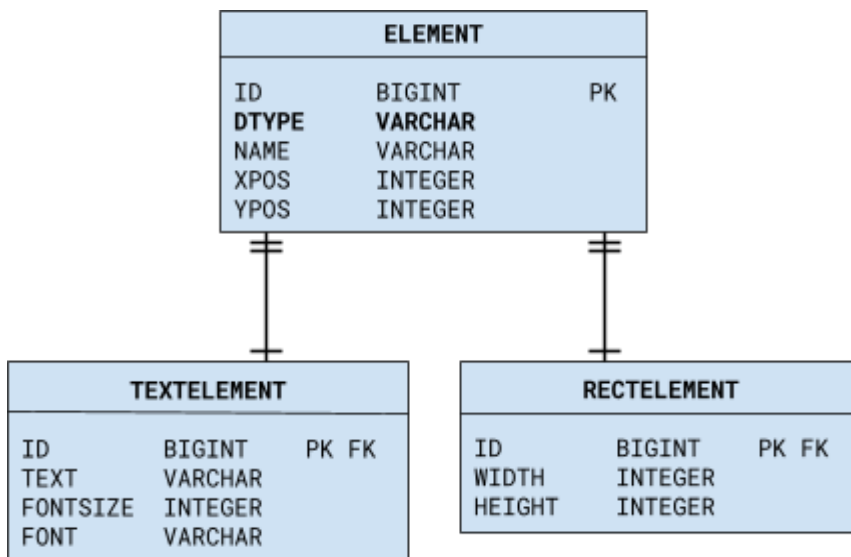
Stratégie mapovania

Štruktúra vygenerovaných tabuliek závisí od zvolenej stratégie mapovania.

SINGLE_TABLE

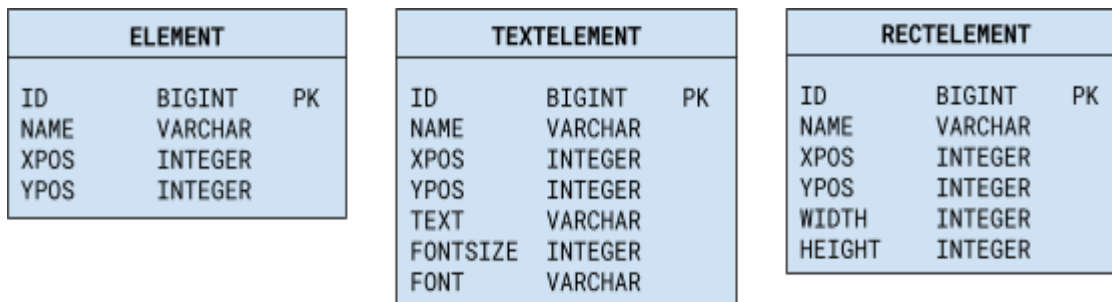
ELEMENT		
ID	BIGINT	PK
DTYPE	VARCHAR	
NAME	VARCHAR	
XPOS	INTEGER	
YPOS	INTEGER	
TEXT	VARCHAR	
FONTSIZE	INTEGER	
FONT	VARCHAR	
WIDTH	INTEGER	
HEIGHT	INTEGER	

JOINED



Poznámka: V oboch stratégiách stĺpec **DTYPE** obsahuje informáciu o triede entitného objektu.

TABLE_PER_CLASS



Poznámka: Tu je trieda entitného objektu je určená priamo tabuľkou.

Mapovanie abstraktnej rodičovskej triedy

Ak je niektorá trieda v hierarchii abstraktná trieda, nie je možné vytvárať jej inštancie a teda nie je ani potrebné vytvárať pre ňu tabuľku. V prípade stratégie TABLE_PER_CLASS sa ani nevytvorí.

V prípade stratégie JOINED sa môžeme rozhodnúť, či vytvoríme tabuľku aj pre abstraktnú triedu alebo jej atribúty namapujeme do tabuliek jej podtried. V takom prípade použijeme namiesto @Entity anotáciu @MappedSuperclass.

```
@MappedSuperclass
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Element implements Serializable {
    @Id
    private Long id;
    private String name;
    private int xpos ;
    private int ypos ;
}
```

Poznámka: v tomto príklade je abstraktný priamo koreň hierarchie ale mohla by to byť aj iná trieda.

Relačný diagram

Atribúty abstraktnej triedy sa presunuli do tabuliek podtried.

TEXTELEMENT		
ID	BIGINT	PK
NAME	VARCHAR	
XPOS	INTEGER	
YPOS	INTEGER	
TEXT	VARCHAR	
FONTSIZE	INTEGER	
FONT	VARCHAR	

RECTELEMENT		
ID	BIGINT	PK
NAME	VARCHAR	
XPOS	INTEGER	
YPOS	INTEGER	
WIDTH	INTEGER	
HEIGHT	INTEGER	

Výber stratégie

Z hľadiska použitia je asi najjednoduchšia stratégia SINGLE_TABLE. Nevýhoda je, že v prípade veľkého počtu atribútov špecifických pre podtriedy, vznikne tabuľka s veľkým počtom stĺpcov, ktoré sú však zväčša prázdne.

Pre rozhodovanie medzi stratégiami JOINED a TABLE_PER_CLASS je rozhodujúci charakter dotazov, aké budú prevažovať pri vyhľadávaní a čítaní údajov z databázy.

Ilustrujú to nasledujúce jednoduché volania metód entity manažera, kde v logu môžeme sledovať vygenerované SQL dotazy.

1. Vyhľadávanie podľa kľúča, ak triedu objektu poznáme

```
TextElement e = em.find(TextElement.class, id);
System.out.println(e.getText());
```

V prípade stratégie TABLE_PER_CLASS metóda find vygeneruje jednoduchý SQL-dotaz, ktorý môžeme vidieť v logu:

```
--SELECT ID, FONT, FONTSIZE, NAME, TEXT, XPOS, YPOS FROM TEXTELEMENT WHERE (ID = ?)
```

V prípade stratégie JOINED je potrebný join niekoľkých tabuliek:

```
--SELECT t0.ID, t0.DTYPE, t0.NAME, t0.XPOS, t0.YPOS, t1.ID, t1.FONT, t1.FONTSIZE, t1.TEXT FROM ELEMENT t0,
TEXTELEMENT t1 WHERE ((t0.ID = ?) AND ((t1.ID = t0.ID) AND (t0.DTYPE = 'TextElement')))
```

2. Vyhľadávanie podľa kľúča, keď triedu objektu nepoznáme

```
Element e = em.find(Element.class, id);
System.out.println(e);
// skusme ci je to TextElement
if (e instanceof TextElement) {
    System.out.println("'" + ((TextElement) e).getText());
}
```

V prípade stratégie TABLE_PER_CLASS musí metóda find postupne dotazovať viaceré tabuľky, keďže nepoznáme konkrétnu triedu objektu a teda nevieme, v ktorej tabuľke sa záznam nájde:

```
--SELECT ID, HIGHT, NAME, WIDTH, XPOS, YPOS FROM RECTELEMENT WHERE (ID = ?)
--SELECT ID, FONT, FONTSIZE, NAME, TEXT, XPOS, YPOS FROM TEXTELEMENT WHERE (ID = ?)
```

V prípade stratégie JOINED si najprv samostatným dotazom zistí dátový typ.

```
--SELECT DISTINCT DTYPE FROM ELEMENT WHERE (ID = ?)
--SELECT t0.ID, t0.DTYPE, t0.NAME, t0.XPOS, t0.YPOS, t1.ID, t1.FONT, t1.FONTSIZE, t1.TEXT FROM ELEMENT t0,
TEXTELEMENT t1 WHERE ((t0.ID = ?) AND ((t1.ID = t0.ID) AND (t0.DTYPE = 'TextElement')))
```

Pri všetkých troch stratégiách si dokáže metóda `find` zistiť skutočnú triedu dotazovanej entity a načítať hodnoty všetkých dátových členov.

3. Načítanie vybraných údajov zo všetkých objektov

Napríklad načítanie súradníc všetkých elementov, môžeme urobiť priamo **pomocou SQL** dotazu.

V prípade stratégie JOINED stačí jednoduchý dotaz:

```
SELECT XPOS, YPOS FROM ELEMENT
```

V prípade stratégie TABLE_PER_CLASS je však potrebný UNION viacerých dotazov.

```
SELECT XPOS, YPOS FROM ELEMENT
UNION (SELECT XPOS, YPOS FROM TEXTELEMENT)
UNION (SELECT XPOS, YPOS FROM RECTELEMENT)
```

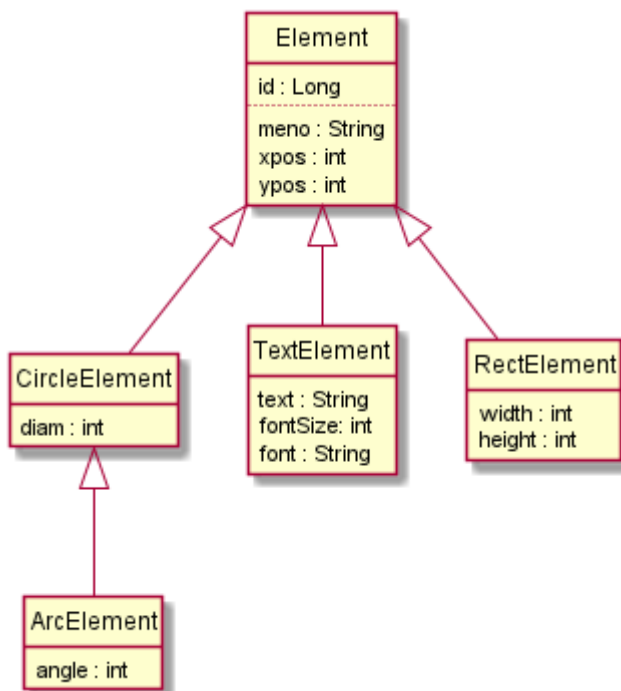
4. Načítanie všetkých údajov zo všetkých objektov

môžeme urobiť jednoduchý JPQL-dotazom, rovnakým pre všetky stratégie:

```
Query q = em.createQuery("select e from Element e");
List<Element> elements = q.getResultList();
for (Element e : elements) {
    System.out.println(e.getName() + " " + e.getClass().toString());
}
```

Cvičenia

Cvičenie 1. Doplňte hierarchiu tried v projekte [p61](#) z prednášky o ďalšiu úroveň podľa UML diagramu.



Vo funkcii `create` vytvorte inštancie všetkých tried a pozrite si štruktúru vygenerovaných tabuliek

- pre všetky stratégie
- pre stratégiu JOINED pričom trieda `Element` bude abstraktná
- pre stratégiu JOINED pričom trieda `CircleElement` bude abstraktná

Cvičenie 2. Pre každú zo stratégií mapovania tried z projektu [p61](#) napíšte SQL dotazy pre načítanie týchto dát resp. objektov.

- počtu všetkých elementov (včítane podtried)
- počtu všetkých textových elementov

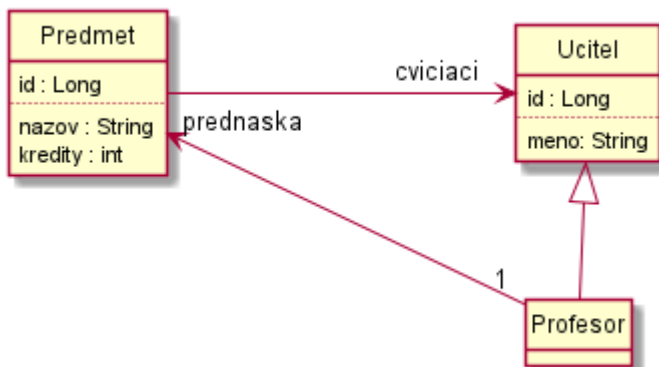
- mien všetkých textových elementov
- všetkých textových elementov
- všetkých elementov s xpos > 0 (včítane podtried)
- všetkých textových elementov s xpos > 0

Cvičenie 3. Pre SQL-dotazy z predchádzajúceho cvičenia skúste nájsť vhodné JPQL alternatívy. Overte funkčnosť dotazov pre jednotlivé stratégie a v logu pozrite vygenerované SQL-dotazy. Pozri dokumentáciu k JPQL napr.

- <https://jakarta.ee/learn/docs/jakartaee-tutorial/current/persist/persistence-querylanguage/persistence-querylanguage.html>
- https://docs.oracle.com/cd/E15523_01/apirefs.1111/e13946/ejb3_langref.html
- http://www.java2s.com/Tutorials/Java/JPA/4000_JPA_Query_Intro.htm

Cvičenie 4.

a) Navrhните a nakreslite štruktúru databázy pre entitné triedy znázornené UML diagramom. Urobte verzie návrhu pre každú zo stratégií mapovania hierarchie tried.

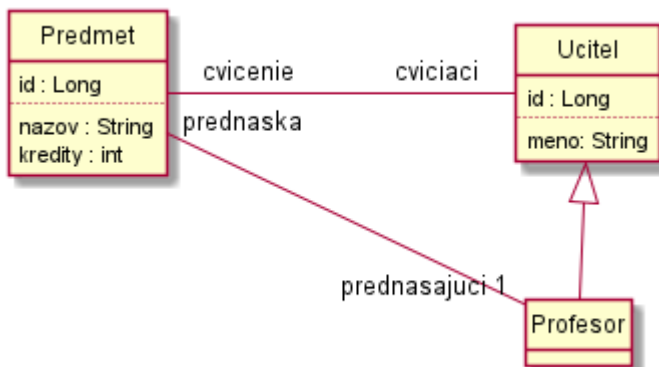


b) Implementujte entitné triedy podľa UML diagramu a anotujte ich tak aby sa namapovali na databázu ako ste ju navrhli v časti a)

c) Naplňte databázu dátami pre testovanie (aspoň jeden predmet s prednášajúcim a niekoľkými cvičiacimi). Implementujte aplikáciu, ktorá z databázy načíta všetky údaje o predmetoch, pustíte ju a overte sú všetky dátové členy objektov a asociácie medzi nimi správne načítané a vytvorené.

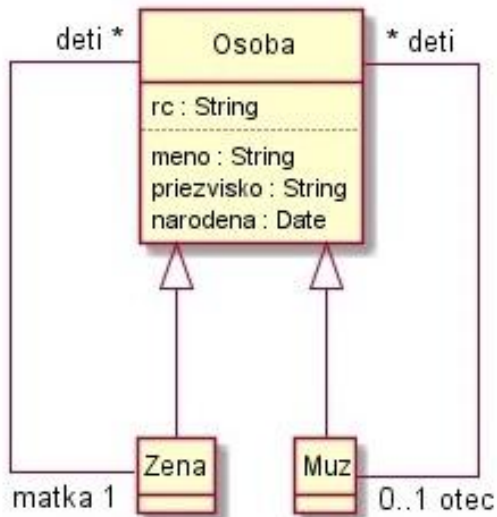
d) Implementujte funkciu, ktorá vráti zoznam všetkých prednášajúcich, ktorí si svoj predmet sami cvičia (sú medzi z cvičiacimi predmetu, ktorý sami prednášajú).

Cvičenie 5. Riešte predchádzajúcu úlohu pre verziu UML diagramu s obojsmernými asociáciami.



Cvičenie 6.

Na obrázku je verzia UML diagramu z [cvičenia 3 časti JPA-4](#), ktorá zachytáva skutočnosť, že matka je žena a otec je muž.



Pre každú zo stratégií mapovania hierarchie najprv načrtnite, ako by vyzerala databáza odpovedajúca UML diagramu. Následne implementujte entitnú triedu podľa UML diagramu a použite JPA anotácie tak, aby z nej JPA vygenerovalo databázu podľa vášho návrhu. Implementujte tiež vhodný program na testovanie.